

Curs 9-10

Diferențe C# si Java

Resurse: [Udemy](#)

C-Sharp. This language was developed to compete with Java, by Microsoft under the leadership of Anders Hejlsberg and his team.

Tipurile de date și clasele de bază

-slide 8

- In **C#** toate tipurile de date sunt derivate din clasa `System.Object` (slide 8)
- În **Java**, tipurile primitive nu derivă din nici o clasă de bază.

Constante

-slide 13

- **C#:** Constantele sunt declarate cu `const` și trebuie inițializate la momentul definirii.
 - `const string Message = "Hello World";`
- **Java:** Constantele sunt declarate cu `final`, iar inițializarea poate fi amânată până la construcție.
 - `final String message;`
`message="Hello World";`

Parametri prin referință

-slide 20

- **C#:** Parametrii pot fi transmiși prin referință folosind cuvintele cheie `ref` și `out`.

```
void Swap(ref int a, ref int b) {
    int temp = a;
    a = b;
    b = temp;
}
int x = 1, y = 2;
Swap(ref x, ref y);
Console.WriteLine($"x = {x}, y = {y}"); // Output: x = 2, y = 1
```

- ****Java:**** Parametrii sunt trecuți prin valoare, dar obiectele pot fi modificate prin referințe.

```
``java
void swap(int[] arr) {
    int temp = arr[0];
    arr[0] = arr[1];
    arr[1] = temp;
}
int[] arr = {1, 2};
swap(arr);
System.out.println(arr[0] + ", " + arr[1]); // Output: 2, 1
```

Transmiterea parametrilor prin referinta: ref si out

- ref
- Trebuie precizat si la definitia functiei si la apel
- Argumentul poate fi de tip valoare sau de tip referinta
- Parametrul trebuie sa fie initializat
- out
- Trebuie precizat si la definitia functiei si la apel
- Cauzeaza transmiterea prin referinta
- Nu impune initializarea paramentruului actual

Accesarea proprietatilor - getter & setter

- **C#**: Are suport nativ pentru proprietăți (get / set). -> comparatie cu C/C++
- dacă nu este nevoie de logică suplimentară în metodele **get** și **set**, se pot utiliza proprietăți automate.

```
class Person {  
    public string Name { get; set; }  
}  
Person p = new Person();  
p.Name = "John";  
Console.WriteLine(p.Name); // Output: John
```

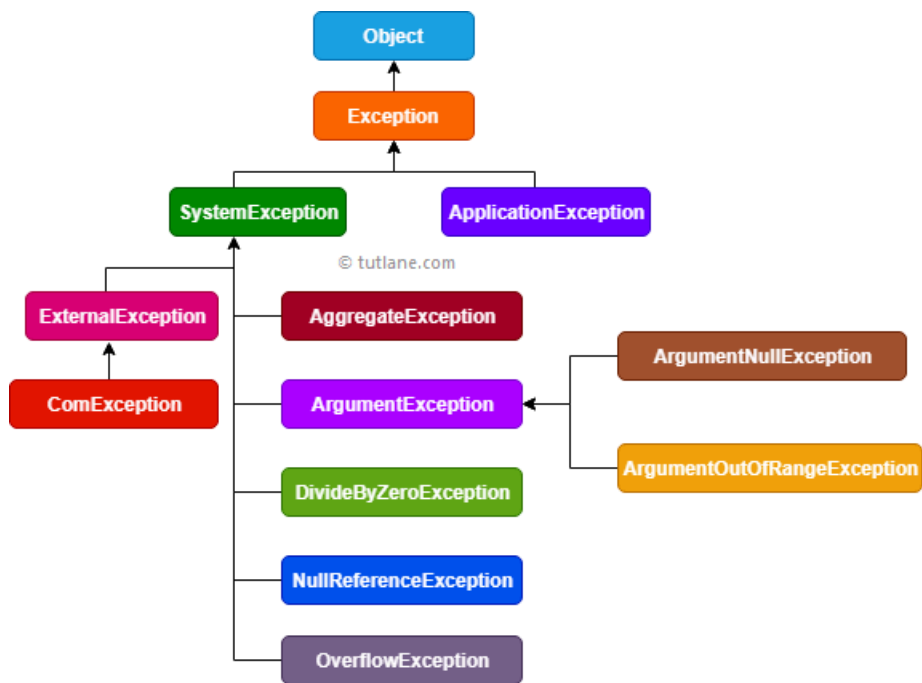
- **Java**: Getter și setter trebuie definite manual.

```
class Person {  
    private String name;  
    public String getName() {  
        return name; }  
    public void setName(String name) {  
        this.name = name; }  
}  
Person p = new Person();  
p.setName("John");  
System.out.println(p.getName()); // Output: John
```

Exceptii

-slide 37

- **C#**: Toate excepțiile derivă din clasa de bază `System.Exception`.
- Nu există conceptul de **checked exceptions**. Toate excepțiile sunt **runtime exceptions** și nu trebuie declarate sau tratate explicit în cod.



```

using System;
using System.IO;

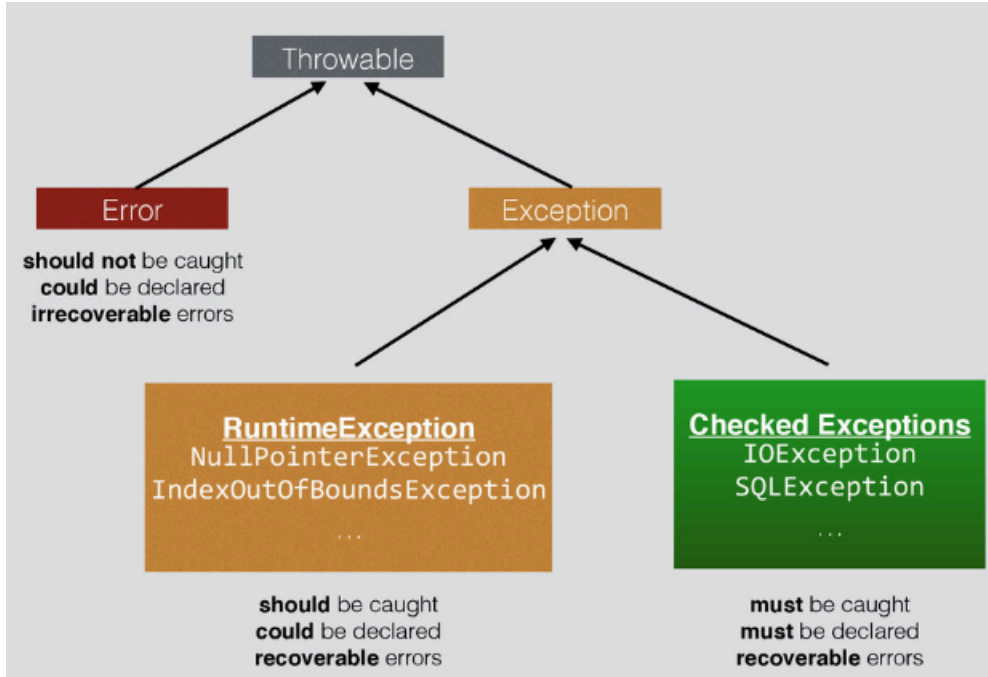
class Test {
    public void ReadFile() {
        StreamReader file = new StreamReader("file.txt"); // IOException
    }

    public static void Main(string[] args) {
        Test test = new Test();
        try {
            test.ReadFile();
        } catch (IOException e) { //a nu se trata
            Console.WriteLine(e.Message);
        }
    }
}

```

- **Java:** Toate excepțiile derivă din clasa de bază `Throwable`, care are două ramuri principale:
 - `Error` (excepții critice, care nu ar trebui tratate)
 - `Exception` (excepțiile care pot fi tratate)
 Distinge între **checked exceptions** și **unchecked exceptions**:
- **Checked exceptions:** Trebuie fie să fie tratate (`try-catch`), fie declarate în semnătura metodei cu `throws`.

- **Unchecked exceptions:** Pot fi ignorate, deoarece sunt moștenite din `RuntimeException`.



```
import java.io.*;

public class Test {
    public void readFile() throws IOException {
        FileReader file = new FileReader("file.txt"); // Checked exception
    }

    public static void main(String[] args) {
        Test test = new Test();
        try {
            test.readFile();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```